

Ouvidoria Inteligente — Busca Semântica de Manifestações

Gabriel Cavalcante Oliveira - Unipê - Ciência da computação

1. Escopo deste relatório

Este documento resume as decisões, dificuldades e aprendizados do buscador semântico de manifestações da ouvidoria. Foi produzido junto com o aplicativo **app_ouvidoria.py** (Entrega 4). Os três notebooks (**análise_comparativa**, **deteccao_duplicatas** e **chunking_manifestacoes**) **não fazem parte deste pacote**. Além disso, o dataset oficial das 40 manifestações não estava disponível quando o app foi construído. Por isso o app traz 40 manifestações **fictícias** de exemplo, escritas só para testar as funcionalidades (3 pares de duplicatas semânticas, 5 textos com mais de 500 caracteres, 5 categorias), e aceita o CSV oficial por upload (colunas id, categoria e texto). Todos os números abaixo vêm dessa base de exemplo e **devem ser refeitos com o dataset oficial e com o modelo de embedding real** antes da entrega.

2. Arquitetura do aplicativo

Componente	Como foi feito	Requisito atendido
Sidebar	Seleção do modelo de embedding (MiniLM multilíngue, BGE-small-pt, mpnet multilíngue ou fallback esperso), slider de top-k (1 a 10) e upload do CSV.	Seleção de modelo e top-k configurável.
Aba Busca Semântica	A consulta é codificada e comparada por cosseno com a base. Retorna o top-k em cartões coloridos: verde (score > 0,7), amarelo (> 0,5) e cinza (demais).	Top-5 com score e destaque por cor.
Aba Base Completa	Tabela de todas as manifestações e botão que gera o heatmap da matriz de similaridade. Abaixo dele, lista de possíveis duplicatas com limiar fixo (padrão 0,85) ou dinâmico (percentil 90).	Tabela e matriz de similaridade.
Aba Espaço Vetorial	Projeção 2D com PCA ou t-SNE, colorida por categoria. Duas métricas objetivas ajudam a comentar os clusters: % de manifestações cujo vizinho mais próximo é da mesma categoria e silhueta por categoria.	Visualização 2D com comentário sobre clusters x categorias.
Aba Chunking	Cola-se ou escolhe-se um texto longo, seleciona-se a estratégia (Recursive do LangChain, tamanho fixo ou por sentenças) e ajustam-se chunk_size e overlap. Mostra chunks, similaridade com o texto completo e com o chunk anterior, embeddings e um PCA 2D.	Chunking com embeddings dos chunks.
Desempenho	st.cache_resource para o modelo e o vetorizador; st.cache_data para embeddings e reduções 2D. Vetores normalizados, então o cosseno é um produto interno.	Dicas do enunciado.

3. Decisões de projeto

Fallback esperso. Se o *sentence-transformers* ou o modelo não puder ser carregado (por exemplo, sem internet na primeira execução), o app usa TF-IDF de n-gramas de caracteres e avisa na sidebar. Assim ele nunca quebra, mas esse modo **não captura sinônimos** e serve só como rede de segurança. O modelo padrão é o *paraphrase-multilingual-MiniLM-L12-v2*, por ser multilíngue e leve. Não consegui verificar se o identificador *BAAI/bge-small-pt-v1.5*, citado no enunciado, existe no Hugging Face; se não carregar, o app cai no fallback com aviso.

Limiar de duplicatas. A função **detectar_duplicatas(textos, limiar=0.85)** devolve todos os pares acima do limiar. Como a escala do cosseno muda de modelo para modelo, o app também oferece o limiar dinâmico (percentil 90 das similaridades entre todos os pares), sugerido no enunciado. A justificativa final do valor precisa ser feita comparando com as duplicatas reais do dataset oficial.

Chunking. Usa o **RecursiveCharacterTextSplitter** do LangChain quando instalado; senão, uma implementação própria do mesmo algoritmo (separadores `\n\n`, `\n`, espaço e caractere). O overlap é limitado a menos que o chunk_size. Para medir a coesão, o app calcula o cosseno entre chunks consecutivos.

4. Resultados preliminares (base de exemplo)

Entrega 1 (parcial). Similaridade de cosseno dos três pares do enunciado, calculada com as representações esparsas na base de exemplo. A coluna de embeddings densos ficou pendente, pois o modelo não pôde ser baixado no ambiente de teste.

Par	BoW	TF-IDF (palavras)	TF-IDF (caracteres)	Embedding denso
M003 × M017 (buraco × asfalto esburacado)	0,000	0,000	0,085	a preencher
M008 × M022 (sem médico × falta atendimento no PSF)	0,201	0,146	0,190	a preencher
M008 × M031 (sem médico × lâmpada queimada)	0,055	0,025	0,041	a preencher

As representações esparsas confirmam a limitação esperada: M003 × M017 descrevem o mesmo problema sem compartilhar nenhuma palavra (BoW e TF-IDF de palavras = 0), e o par não relacionado M008 × M031 fica baixo. Já o TF-IDF de caracteres capta um pouco de afinidade morfológica (0,085), mas continua longe do que se espera de um embedding denso para duplicatas semânticas. **Isso é uma expectativa a ser confirmada** com o modelo

real, e não um resultado medido.

Observações do fallback. Na busca “asfalto cheio de buracos na rua”, o fallback colocou M017 (0,24) e M003 (0,15) nas duas primeiras posições, mas com scores baixos (nenhum passa de 0,5, então todos os cartões ficam cinza). O vizinho mais próximo pertence à mesma categoria em 37,5% das manifestações, e com limiar 0,3 só o par M012 × M035 foi detectado como duplicata. Ou seja, os limiares 0,7 e 0,85 do enunciado são pensados para embeddings densos e não se aplicam ao fallback.

Overlap no chunking. Em M005 com `chunk_size = 200`, o chunk 2 começa em “veio consertar...” com overlap 0 e em “o corrimão caiu há dois meses e ninguém...” com overlap 60, ou seja, o overlap repete o final do chunk anterior e reduz o corte abrupto de ideias. Qual configuração preserva melhor o sentido das denúncias ainda precisa ser respondida com o modelo real e com a comparação de pelo menos duas configurações.

5. Dificuldades

(i) O ambiente de desenvolvimento não tinha internet, então não foi possível baixar os modelos nem instalar *streamlit*, *sentence-transformers* e *langchain*; a lógica e as funções de cada aba foram testadas com um Streamlit simulado, mas **a interface não foi aberta no navegador**. (ii) A ausência do dataset oficial impediu calcular as análises pedidas (falsos positivos e negativos das duplicatas, comparação de clusters com categorias). (iii) O t-SNE exige perplexidade menor que o número de amostras, o que num corpus de 40 documentos obriga a usar um valor pequeno, e a projeção varia bastante com esse parâmetro. (iv) O cache do Streamlit exige argumentos “hasheáveis”, por isso as listas de textos são passadas como tuplas.

6. Aprendizados e próximos passos

Representações densas trocam a precisão lexical pela cobertura semântica, e por isso os limiares de similaridade dependem do modelo e não devem ser copiados entre eles. Normalizar os vetores simplifica o cálculo e o cache. O chunking é um compromisso entre contexto (chunks maiores, mais overlap) e especificidade (chunks menores). Próximos passos: (1) rodar o app com o dataset oficial e o modelo real; (2) preencher a coluna de embeddings da tabela da seção 4; (3) produzir os três notebooks; (4) definir o limiar de duplicatas comparando com as duplicatas reais e medir falsos positivos e negativos; (5) comparar pelo menos dois modelos e duas configurações de chunking; (6) preencher os nomes da equipe.